Task - 3: Group Task
Project: Titanic Passenger Data Analysis:

Business scenario:

Business problem: A maritime insurance and passenger-safety team wants to analyze historical Titanic passenger data to understand passenger demographics, fare distribution, and survival outcomes. The team wants to use numerical analysis to identify patterns and support future passenger-safety planning.

DATASET:

[1]:
```
import numpy as np
import pandas as pd

df = pd.read_csv("train.csv")

df.head()
```

[1]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

[2]:
```
df.shape
```

[2]:
```
(891, 12)
```

1. NumPy — Numerical analysis:

Analyze passenger ages and fares using arrays, calculate statistical measures, handle missing ages using the mean, and identify unusually high fares.

1.1 What do passenger age and fare statistics tell us about the Titanic passenger population, and how can we handle missing numerical values?

STEPS:

1.Converting passenger ages into a NumPy array and identifying missing values in the Titanic dataset.

[3]:
```
import numpy as np
import pandas as pd

df = pd.read_csv("train.csv")

print("Dataset shape:", df.shape)
print("Passenger ages missing:", df["Age"].isna().sum())
print("Passenger fares missing:", df["Fare"].isna().sum())
```

```
Dataset shape: (891, 12)
Passenger ages missing: 177
Passenger fares missing: 0
```

[ ]:
```
1 – Convert passenger ages into an array:
```

[4]:
```
ages = df["Age"].to_numpy()

print(ages[:10])
print("Array type:", type(ages))
print("Total passengers:", len(ages))
```

```
[22. 38. 26. 35. 35. nan 54.  2. 27. 14.]
Array type: <class 'numpy.ndarray'>
Total passengers: 891
```

2. Calculating basic age statistics such as mean, median, minimum, maximum, and standard deviation.

```
[ ]: 2 – Calculate basic statistics
```

```
[5]: valid_ages = ages[~np.isnan(ages)]

print("Mean age:", np.mean(valid_ages))
print("Median age:", np.median(valid_ages))
print("Youngest passenger:", np.min(valid_ages))
print("Oldest passenger:", np.max(valid_ages))
print("Age standard deviation:", np.std(valid_ages))
```

```
Mean age: 29.69911764705882
Median age: 28.0
Youngest passenger: 0.42
Oldest passenger: 80.0
Age standard deviation: 14.516321150817316
```

3. Handling missing age values using mean imputation, replacing NaN values with the rounded mean age of 30.

```
[ ]: 5 – Replace missing ages with the mean
```

```
[8]: ages_cleaned[np.isnan(ages_cleaned)] = mean_age
```

```
[11]: ages = df["Age"].to_numpy()

mean_age = round(np.nanmean(ages))

ages_cleaned = ages.copy()
ages_cleaned[np.isnan(ages_cleaned)] = mean_age

print("Mean age used:", mean_age)
print("Missing before:", np.isnan(ages).sum())
print("Missing after:", np.isnan(ages_cleaned).sum())
```

```
Mean age used: 30
Missing before: 177
Missing after: 0
```

4. Classifying passengers into age groups to count children, adults, and seniors.

```
[ ]: 4 – Identify passengers in different age groups
```

```
[7]: children = np.sum(ages_cleaned < 18)

adults = np.sum(
    (ages_cleaned >= 18) & (ages_cleaned < 60)
)

seniors = np.sum(ages_cleaned >= 60)

print("Children:", children)
print("Adults:", adults)
print("Seniors:", seniors)
```

```
Children: 113
Adults: 752
Seniors: 26
```

5. Recalculating age statistics after cleaning, rounding all ages to whole numbers for easier interpretation.

```
[ ]: 6 – Analyze the cleaned ages
```

```
[13]: # Calculate mean ignoring NaN values and round it
mean_age = round(np.nanmean(ages))

# Replace missing ages with the rounded mean
ages_cleaned = df["Age"].to_numpy().copy()
ages_cleaned[np.isnan(ages_cleaned)] = mean_age

# Round every age to a whole number
ages_cleaned = np.round(ages_cleaned).astype(int)

# Calculate statistics
print("Mean age:", round(np.mean(ages_cleaned)))
print("Median age:", round(np.median(ages_cleaned)))
print("Minimum age:", np.min(ages_cleaned))
print("Maximum age:", np.max(ages_cleaned))
print("Standard deviation:", round(np.std(ages_cleaned)))
```

```
Mean age: 30
Median age: 30
Minimum age: 0
Maximum age: 80
Standard deviation: 13
```

2. PANDAS — Pandas mini-project: Titanic Passenger Data Analysis:

Business scenario

Passenger Safety Analysis: A maritime safety team wants to analyze passenger records to understand survival rates by passenger class and gender, identify missing information, and determine which passenger groups had lower survival rates.

STEPS:

1. Load and Inspect: Import Pandas, load train.csv, and inspect the dataset's dimensions, columns, and data types.

```
[ ]: Task 1 – Load and inspect the dataset
```

```
[21]: import pandas as pd

df = pd.read_csv("train.csv")

print(df.shape)
print(df.info())
```

```
(891, 12)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None
```

2. Identify Missing Values: Use isnull().sum() to identify missing values in each column.

[ ]:
```
Task 2 – Identify missing values
```

[21]:
```
import pandas as pd

df = pd.read_csv("train.csv")

print(df.shape)
print(df.info())
```

```
(891, 12)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   PassengerId  891 non-null    int64  
 1   Survived     891 non-null    int64  
 2   Pclass       891 non-null    int64  
 3   Name         891 non-null    object 
 4   Sex          891 non-null    object 
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64  
 7   Parch        891 non-null    int64  
 8   Ticket       891 non-null    object 
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object 
 11  Embarked     889 non-null    object 
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None
```

[15]:
```
print(df.isnull().sum())
```

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
```

3. Clean Missing Values: Fill missing ages with the rounded mean, missing embarkation ports with the mode, and missing cabins with "Unknown".

4. Survival by Gender: Group passengers by gender and calculate their survival percentages.

5. Survival by Class: Group passengers by ticket class and compare their survival rates.

[ ]:
```
Task 3 – Clean missing values
```

[16]:
```
df["Age"] = df["Age"].fillna(round(df["Age"].mean()))
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
df["Cabin"] = df["Cabin"].fillna("Unknown")

print(df.isnull().sum())
```

```
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin          0
Embarked       0
dtype: int64
```

[ ]:
```
Task 4 – Calculate survival rates by gender
```

[17]:
```
survival_gender = df.groupby("Sex")["Survived"].mean() * 100

print(survival_gender.round(2))
```

```
Sex
female    74.20
male      18.89
Name: Survived, dtype: float64
```

[ ]:
```
Task 5 – Compare survival by passenger class
```

[18]:
```
survival_class = df.groupby("Pclass")["Survived"].mean() * 100

print(survival_class.round(2))
```

```
Pclass
1    62.96
2    47.28
3    24.24
Name: Survived, dtype: float64
```

6. Filter Non-Survivors: Filter passengers who did not survive and display their names, gender, age, and class.

7. Average Fare by Class: Group passengers by ticket class and calculate the average fare for each class.

[ ]:
```
Task 6 – Filter passengers who did not survive
```

[19]:
```
not_survived = df[df["Survived"] == 0]

print(not_survived[["Name", "Sex", "Age", "Pclass"]].head(10))
print("Total non-survivors:", len(not_survived))
```

```
                                             Name     Sex   Age  Pclass
0                           Braund, Mr. Owen Harris    male  22.0       3
4                         Allen, Mr. William Henry    male  35.0       3
5                                 Moran, Mr. James    male  30.0       3
6                          McCarthy, Mr. Timothy J    male  54.0       1
7                   Palsson, Master. Gosta Leonard    male   2.0       3
12                  Saundercock, Mr. William Henry    male  20.0       3
13                     Andersson, Mr. Anders Johan    male  39.0       3
14             Vestrom, Miss. Hulda Amanda Adolfina  female  14.0       3
16                             Rice, Master. Eugene    male   2.0       3
18  Vander Planke, Mrs. Julius (Emelia Maria Vande...  female  31.0       3
Total non-survivors: 549
```

[ ]:
```
Task 7 – Find the average fare by passenger class
```

[20]:
```
average_fare = df.groupby("Pclass")["Fare"].mean()

print(average_fare.round(2))
```

```
Pclass
1    84.15
2    20.66
3    13.68
Name: Fare, dtype: float64
```

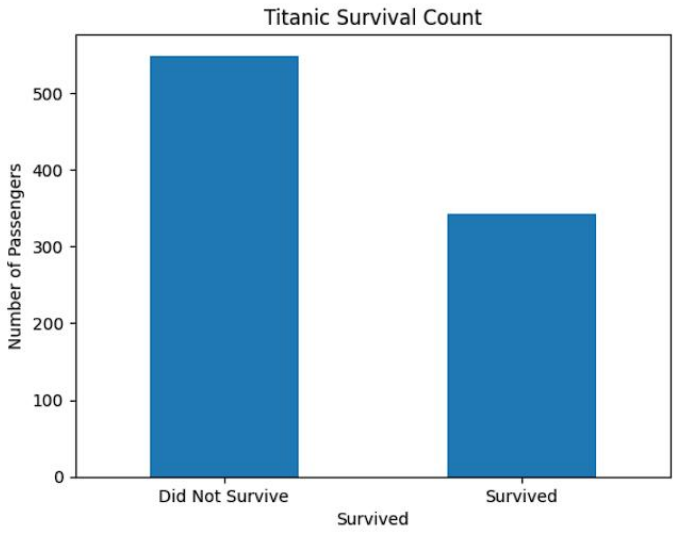
3.MATPLOTLIB— Pictorial Analysis of Titanic Data:

Business scenario: A maritime safety team wants to visualize passenger survival rates, age distribution, and ticket fares to understand historical passenger patterns.

1. Visualized the total number of Titanic survivors and non-survivors.

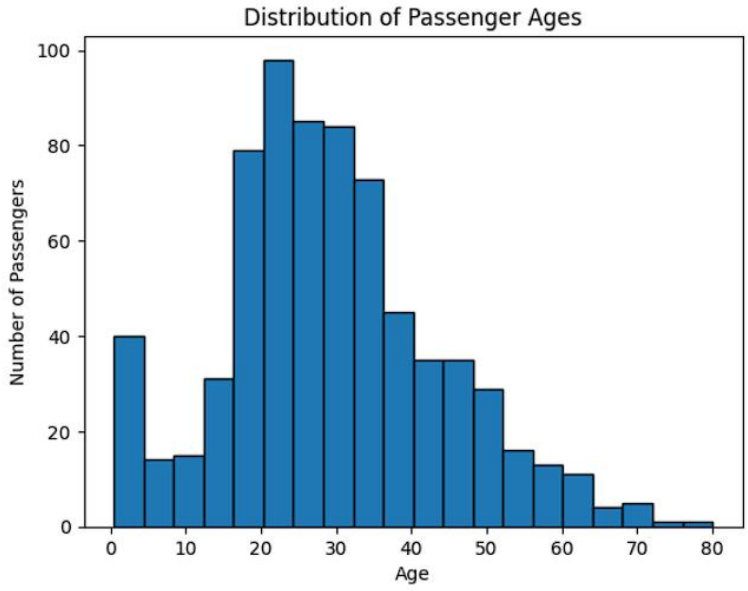
```
[ ]: Task 1 – Survival count
```

```
[23]: df["Survived"].value_counts().sort_index().plot(
          kind="bar", title="Titanic Survival Count"
      )
      plt.xticks([0, 1], ["Did Not Survive", "Survived"], rotation=0)
      plt.ylabel("Number of Passengers")
      plt.show()
```



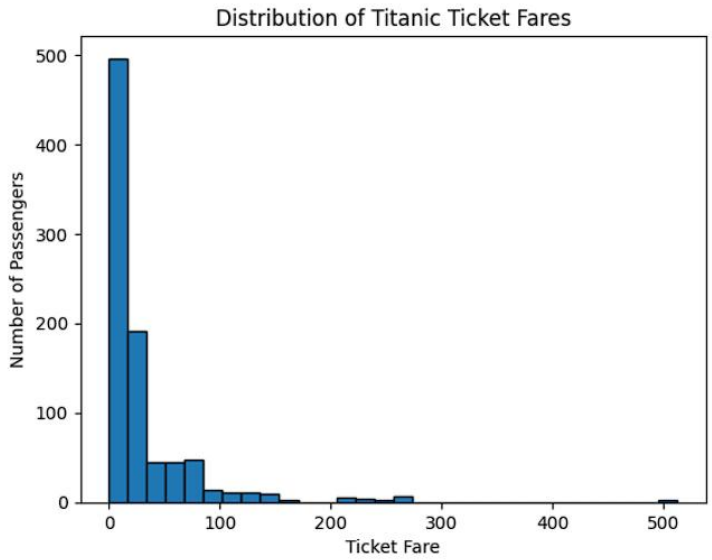
2. Analyzed passenger age distribution using a histogram.

```
[24]: plt.hist(df["Age"].dropna(), bins=20, edgecolor="black")
      plt.title("Distribution of Passenger Ages")
      plt.xlabel("Age")
      plt.ylabel("Number of Passengers")
      plt.show()
```



3.Investigated ticket fare distribution and potential high-fare outliers.

[25]:
```
plt.hist(df["Fare"], bins=30, edgecolor="black")
plt.title("Distribution of Titanic Ticket Fares")
plt.xlabel("Ticket Fare")
plt.ylabel("Number of Passengers")
plt.show()
```



4.Explored the relationship between passenger age and ticket fare using a scatter plot.

[26]:
```
plt.scatter(df["Age"], df["Fare"], alpha=0.5)
plt.title("Passenger Age vs Ticket Fare")
plt.xlabel("Age")
plt.ylabel("Ticket Fare")
plt.show()
```

